INTERNSHIP DOCUMENTATION REPORT

TASK 1: DATA COLLECTION & DATA SOURCE IDENTIFICATION

Exploratory Data Analysis (EDA) on the Titanic Dataset Using Python Pandas

Internship Domain: Data Analytics / AI & ML

Submitted By: Dhruv Bhaveshbhai Modi

Institution: GEC Patan (GTU)

Date of Submission: May 26, 2026

Tools & Technologies: Python • Pandas • Google Colab • GitHub • CSV Dataset

Abstract

This documentation outlines the foundational project of the Data Analytics internship, focusing on data collection, source identification, and initial structural profiling. The core objective is to perform comprehensive Exploratory Data Analysis (EDA) on the standard Titanic Dataset using Python's Pandas library within Google Colab. The project establishes structural awareness of tabular data, maps columns to discrete mathematical data types, identifies and quantifies missing data entries, and verifies unique category distributions. Ultimately, this report details the step-by-step procedural workflows, technical concepts, line-by-line code implementations, and key learning outcomes vital for executing dependable data preprocessing prior to predictive machine learning.

Table of Contents

1. Introduction

What is Data Analysis?

Data analysis is the systematic process of inspecting, cleaning, transforming, and modeling raw data with the overarching goal of discovering useful information, suggesting conclusions, and supporting operational decision-making. In modern computing, raw numbers, text, and categories are continuously generated. Without structural interpretation, this data remains an untapped utility. Data analysis translates unstructured or unorganized inputs into rich, actionable insights.

Importance of Data Understanding Before Machine Learning

In the pipeline of predictive data modeling, feeding raw, unexamined data directly into a Machine Learning (ML) algorithm is an anti-pattern that leads to model failure. Algorithms rely on mathematical mathematical patterns; if the underlying data contains errors, structural inconsistencies, unhandled data formats, or extreme distribution shifts, the resulting model will deliver highly flawed predictions. Developing an intimate, logical understanding of data constraints, formatting parameters, and underlying properties is a strict prerequisite for any subsequent algorithmic ingestion.

Importance of Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is an approach to analyzing datasets by summarizing their main visual or mathematical characteristics, frequently employing statistical profiles. EDA allows data practitioners to look closely at what the data contains beyond formal hypothesis testing or predictive modeling frameworks. It assists in uncovering latent data distributions, validating foundational assumptions, diagnosing data anomalies, detecting missing metrics, and isolating variables that contain high predictive power.

Brief Introduction to the Titanic Dataset

The Titanic Dataset is a foundational, globally recognized database containing records of the passengers who boarded the ill-fated R.M.S. Titanic in 1912. It provides individual details regarding passenger demographics (age, sex, family status), trip specifics (ticket numbers, passenger class, cabin location, boarding port), and the ultimate survival status of each individual. It serves as an optimal introductory playground for practicing exploratory data profiling, feature handling, and basic programming syntaxes.

2. Objective

The operational and learning objectives of this initial task are established as follows:

- **Understanding Dataset Structure:** Gaining concrete visibility into how data observations (rows) and feature attributes (columns) align within a unified tabular architecture.
- **Identifying Numerical and Categorical Data:** Dissecting feature attributes to classify them correctly as numerical vectors (continuous or discrete) versus categorical references (nominal or ordinal).

- **Detecting Missing Values:** Investigating the structural health of the array by scanning for null elements, blank blocks, or missing values (NaN) that require eventual engineering remediation.
- **Performing Basic Dataset Inspection:** Executing programmatic diagnostics to measure data shapes, physical memory usage, boundary limits, and baseline index structures.
- **Understanding Structured Datasets:** Adapting to rigid row-by-column structures where data types are highly standardized, consistent, and predictable across individual data lines.

3. Dataset Information

The Titanic Dataset is an educational benchmark distributed heavily on platforms like Kaggle. It acts as an approachable entry point for learning data science because it encapsulates real historical entries that blend rich textual data with structural numeric categories.

The real-world significance of the dataset lies in its historical truth: during the maritime disaster, survival was not entirely random. Distinct social groups, demographic categories, and economic tiers exhibited profoundly different survival rates, reflecting the "women and children first" protocol and socio-economic realities of 1912. Analyzing this dataset teaches students how human behavior, corporate policy, and crisis outcomes translate into rows and columns of structured data.

Dataset Profile and Structure

- **Dataset Type:** Structured / Tabular (Composed of rows and columns)
- **Total Records (Rows):** 891 distinct rows
- **Total Features (Columns):** 12 distinct columns
- **Core Focus Elements:** Passenger identification, class groupings, age vectors, survival tracking flags, and spending parameters.
- **Official Dataset Access Link:** https://www.kaggle.com/datasets/yasserh/titanic-dataset

4. Tools and Technologies Used

The execution of this exploratory data task was powered by the following open-source tools and platforms:

Tool / Technology	Core Purpose	Detailed Application Description
Python	Programming Language	The high-level foundational language used for data manipulation, script processing, and logical data operations.
Pandas	Data Analysis Library	A crucial data manipulation library that provides high-performance data structures like `DataFrame` and `Series` for handling tabular datasets.
Google Colab	Development Environment	An interactive cloud-based Jupyter Notebook environment that allows execution of Python code directly inside web browsers without local configuration.
GitHub	Version Control System	A cloud platform hosting Git software, used to manage code changes, host task repositories, track revision history, and publish documentation.
CSV Dataset	Data Format	Comma-Separated Values file format (`Titanic-Dataset.csv`) storing tabular information as plain text lines separated by commas.

5. Installation and Setup

To establish a clean, repeatable analysis environment, the following procedural steps were performed:

Step 1: Accessing Google Colab

An internet browser was launched, and the user navigated to `colab.research.google.com`. Signing in via an authenticated Google account opened the environment. A new notebook was initialized by clicking *"New Notebook"*, which instantly spun up a secure cloud virtual machine hosted by Google, preconfigured with the latest Python interpreter and core data tools.

Step 2: Uploading the CSV Dataset

The local file `Titanic-Dataset.csv` was prepared. Inside the Google Colab interface, the **Files** icon located on the left-hand navigation sidebar was selected. The *"Upload to session storage"* button was clicked, transferring the local dataset onto the remote cloud container instance.

Step 3: Initializing Libraries and Cell Execution

A code cell was utilized to import the necessary tool libraries. Code execution was triggered by clicking the **Play** button positioned to the left of the active cell or pressing the keyboard combination `Shift + Enter`. This sent code fragments directly to the cloud execution core, outputting structures immediately beneath the input block.

6. Steps Performed

The Exploratory Data Analysis workflow followed a systematic path of progressive discovery:

1. **Importing the Library:** Initialized the Pandas workspace under an standardized programming alias (`pd`).
2. **Loading the Dataset:** Read the plain-text CSV data format into a functional, structured tabular object called a `DataFrame`.
3. **Displaying First Rows:** Printed out the top five rows of the dataset to visually confirm proper formatting, comma separation, and column naming.
4. **Checking Dataset Shape:** Requested index dimensions to verify that all rows (891) and structural attributes (12) were present.
5. **Understanding Data Types:** Consulted structural metadata mapping to view the distribution of integer variables, float variables, and character text definitions.
6. **Using info():** Extracted a comprehensive overview of row counts, non-null values, active feature data types, and operational RAM footprint.
7. **Using describe():** Analyzed statistical boundaries to extract numeric metrics including standard deviations, mean calculations, medians, minimum values, and maximum limits.

8. **Checking Missing Values:** Scanned each feature array to isolate missing rows, building a aggregate list of missing elements.
9. **Identifying Numerical & Categorical Columns:** Categorized features into qualitative groupings versus quantitative measurements based on behavioral properties.
10. **Displaying Unique Values:** Profiled nominal text classes to catalog all unique variations and check for entry noise or inconsistent wording.

7. Explanation of Data Types

In data analysis, data features are defined by their structural and mathematical properties. Understanding these distinct types allows analysts to determine what statistical operations are logically sound for each column.

1. Numerical Data

Numerical data represents quantities, counts, or measurements that are recorded as numbers. These columns support standard mathematical operations such as addition, subtraction, mean calculation, and variance profiling.

- **Continuous Variables:** Variables that can take an infinite number of values within a given range. They represent measurements that can be broken down into finer decimal increments.
  Real-world Example: The weight of cargo, temperature tracking, or financial stock prices.
- **Discrete Variables:** Variables that represent distinct, separate counts that cannot be divided into fractional units. They change only in specific whole-number jumps.
  Real-world Example: The number of children in a household, cars in a parking area, or pages in a document.

2. Categorical Data

Categorical data represents qualitative characteristics, labels, attributes, or distinct groups. These values do not possess intrinsic mathematical weight; instead, they sort observations into distinct groups.

- **Nominal Variables:** Categories that possess no internal sequence, rank, or preference order. They are simply unique named entities.
  Real-world Example: Blood groups (A, B, AB, O), eye color, country names, or manufacturing brand labels.
- **Ordinal Variables:** Categories that exhibit a natural, logical order or hierarchical progression, though the exact numerical distance between choices is not fixed or measurable.
  Real-world Example: Customer satisfaction scores (Poor, Fair, Good, Excellent), academic grades (A, B, C), or military rank hierarchies.

3. Textual Data

Textual data consists of free-form, unstructured strings of characters. Unlike categorical labels, textual data contains high-cardinality values that are often unique to each individual observation. It cannot be directly categorized or measured without advanced processing or feature extraction.
Real-world Example: An individual's full name, custom feedback commentaries, or handwritten address details.

8. Data Types Identified in Titanic Dataset

The 12 features present in the `Titanic-Dataset.csv` were mapped and evaluated against their computational storage formats and data types:

Column Name	Pandas Type	Data Category	Context and Structural Meaning
`PassengerId`	int64	Numerical (Discrete)	A unique sequential serial number assigned to each row to identify a passenger.
`Survived`	int64	Categorical (Nominal)	A binary status flag tracking the outcome (0 = Deceased, 1 = Survived).
`Pclass`	int64	Categorical (Ordinal)	The travel class of the passenger (1 = First Class, 2 = Second Class, 3 = Third Class).
`Name`	object	Textual	Free-form text string capturing the full name, title, and family naming notation.
`Sex`	object	Categorical (Nominal)	The binary biological sex classification of the passenger (male or female).
`Age`	float64	Numerical (Continuous)	The physical age of the person. Includes fractional decimals for infants.
`SibSp`	int64	Numerical (Discrete)	A count of siblings and spouses traveling alongside the passenger.
`Parch`	int64	Numerical (Discrete)	A count of parents and children traveling alongside the passenger.
`Ticket`	object	Textual / Mixed	The ticket identifier string containing mixed alphanumeric tracking codes.
`Fare`	float64	Numerical (Continuous)	The amount of money paid for the travel ticket, mapped as a decimal currency value.
`Cabin`	object	Categorical (Nominal)	The alphanumeric identifier designation of the room block where a passenger was assigned.
`Embarked`	object	Categorical (Nominal)	The single-letter code for the port where the passenger boarded (C, Q, or S).

9. Missing Value Analysis

What are Missing Values?

Missing values, standardly designated as `NaN` (Not a Number) or `Null` entries within data structures, represent blank zones or unrecorded metrics where no information was collected for that attribute during data gathering.

Why Handling Missing Values is Crucial

Missing cells disrupt computational workflows. Many mathematical toolkits and statistical models fail or throw explicit errors if input arrays contain missing data. Ignoring missing values can bias analyses, as the missingness itself often follows a non-random trend that distorts the sample's true properties.

Analysis of Missingness in the Titanic Dataset

Running programmatic inspections revealed that out of 891 entries, three columns contained missing values:

- **Age (177 Missing Entries):** Roughly 19.8% of the age metrics are missing. This missingness requires strategy: if we evaluate survival based on youth, filling these cells via mean/median imputation or predictive guessing is necessary.
- **Cabin (687 Missing Entries):** A massive 77.1% of the data rows lack room records. Because more than three-quarters of this column is blank, dropping the column entirely or converting it into a binary "Cabin Assigned vs No Cabin Assigned" flag is typical during feature engineering.
- **Embarked (2 Missing Entries):** Only 2 records are missing. Because this missingness is extremely small (0.22%), these rows can easily be filled using the most frequent boarding port (mode imputation) without biasing the data.

10. Unique Values in Categorical Columns

Isolating unique entries in nominal columns is a powerful technique for auditing data quality. It helps verify category classifications and uncover data-entry bugs before data modeling begins.

In the Titanic dataset, running a unique inquiry on the `Sex` column yields exactly two distinct variations: `['male', 'female']`. This proves the variable is clean, consistent, and completely free of spelling typos or mixed case strings (e.g., 'Male', 'M', or 'FEMALE').

Similarly, checking the `Embarked` port column exposes three distinct operational strings: `['S', 'C', 'Q']` (alongside the expected missing `nan` value). These codes correspond to Cherbourg, Queenstown, and Southampton. By verifying these unique outputs, the analyst confirms that the data aligns with historical constraints, simplifying the translation of these text categories into numeric vectors for machine learning model ingestion.

11. Python Code Explanation

This section provides a line-by-line explanation of the Python syntax implemented during the data inspection task:

1. Importing the Library Workspace

```
import pandas as pd
```

Line-by-Line Explanation: The keyword `import` instructs the Python interpreter to load the external Pandas library toolkit into the local environment memory. The phrase `as pd` establishes a standardized shorthand alias, allowing the analyst to invoke library methods using `pd.method()` instead of retyping the full word.

2. Ingesting Raw Tabular Files

```
df = pd.read_csv('Titanic-Dataset.csv')
```

Line-by-Line Explanation: This command invokes the `read_csv()` parsing tool from Pandas. It searches session storage for the specified file name, reads its comma-separated structure, and converts it into an active, memory-resident `DataFrame` object assigned to the variable name `df`.

3. Visually Inspecting Top-Tier Rows

```
df.head()
```

Line-by-Line Explanation: The `head()` object method displays the first five chronological rows of the active `df` DataFrame by default. This allows the analyst to quickly inspect the parsed rows, columns, and data cells to confirm formatting is correct.

4. Checking Index Matrix Dimensions

```
df.shape
```

Line-by-Line Explanation: The `.shape` attribute returns a simple Python tuple indicating the structural dimensions of the DataFrame, formatted as `(rows, columns)`. Running this command on the Titanic data outputs `(891, 12)`, confirming the file contains 891 records across 12 distinct features.

5. Inspecting Structural Memory Allocation types

```
df.dtypes
```

Line-by-Line Explanation: This attribute prints a list of all column labels alongside their assigned computational storage types (such as `int64` for integers, `float64` for decimals, or `object` for text strings), showing how columns are recognized by the system.

6. Extracting Full Structural Summaries

```
df.info()
```

Line-by-Line Explanation: The `info()` method outputs a comprehensive overview of the dataset's structural health. It prints out total row indices, lists every column name, tallies the number of non-null cells in each column, shows data types, and reports the active RAM storage size used by the DataFrame.

7. Generating Descriptive Statistical Matrix Overview

```
df.describe(include='all')
```

Line-by-Line Explanation: The `describe()` method generates summary statistics for the dataset. Passing the parameter `include='all'` forces the function to profile both numeric columns (calculating mean, standard deviation, min, and max) and categorical columns (reporting unique label counts, top occurring entries, and their frequency).

8. Quantifying Aggregate Column Missingness

```
df.isnull().sum()
```

Line-by-Line Explanation: This chained expression operates in two parts: first, `isnull()` scans every cell in the DataFrame, converting values to `True` if they are missing and `False` if they contain valid data. Then, `sum()` adds up the `True` values column-by-column, providing a clear count of missing values for each feature.

9. Isolation Filtering based on Data Types

```
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

Line-by-Line Explanation: The `select_dtypes()` method filters the DataFrame's columns based on their data types. By setting the include parameter to numerical identifiers, it isolates matching numerical vectors and extracts their column names via the `.columns` attribute.

10. Extracting Unique Label Categories

```
df['Embarked'].unique()
```

Line-by-Line Explanation: This command isolates the single feature column `'Embarked'` from the DataFrame and runs the `unique()` method on it. The method filters out duplicate entries, returning an array of the distinct category labels present in that column.

13. GitHub Repository Section

Version control management was executed by pushing the project files to a public repository on GitHub. This process helps ensure work remains open, traceable, and secure.

1. Repository Creation

The analyst logged into GitHub and clicked the **"New"** repository button. The repository was named `Data-Collection-Data-Source-Identification`. The visibility setting was configured as **Public** to allow review, and the option to initialize with a default `.gitignore` filter was bypassed to maintain a clean workspace configuration.

2. Code Staging and Uploading Files

Within the newly created repository interface, the **"Upload files"** option was chosen. The completed project notebook file (`task_1.ipynb`) and the dataset file (`Titanic-Dataset.csv`) were dragged and dropped into the web staging panel.

3. Committing Changes and Documenting via README.md

To finalize the file upload, descriptive revision notes were added to the commit message input box before clicking **"Commit changes"**. This saved the code to the main branch history. Finally, a structured `README.md` markdown file was generated to serve as the documentation homepage for the project, detailing the project's layout, objectives, and simple code installation instructions.

14. Challenges Faced

During the execution of this initial project, several beginner-level technical challenges were encountered and successfully resolved:

- **Differentiating Data Type Contexts:** Initially, understanding why numerical columns like `Survived` or `Pclass` were grouped as integers by Pandas but operated logically as categorical factors was confusing. This was resolved by researching the difference between physical storage data types and structural mathematical data types.
- **GitHub Authentication and Staging Issues:** Encountered minor workflow snags when staging multiple files simultaneously through the web browser interface. The issue was fixed by clearing cache states and making distinct, incremental commits.
- **Loop Syntax Errors During Column Profiling:** Faced minor syntax and indentation bugs while writing custom loops to isolate unique value arrays. These errors were resolved by using standard Python debugging workflows and referring to the official Pandas documentation.
- **Interpreting Missing Value Traps:** Discovered that missing values in categorical columns sometimes render as empty strings rather than true system `NaN` objects, requiring careful handling during null count profiling.

15. Learning Outcomes

The successful completion of this initial project provided several valuable skills and technical capabilities:

- **Core Pandas Mastery:** Mastered the syntax for importing external tool libraries, building DataFrame structures, and running foundational data exploration methods.
- **Data Architecture Inspection:** Gained hands-on experience examining row-and-column data frames and diagnosing issues like low-quality or missing metrics.
- **Understanding Structural Data Datasets:** Developed the ability to map data column constraints, differentiate numerical variables from categorical labels, and identify data anomalies.
- **Handling Missing Data:** Learned how to use programmatic expressions to find missing data vectors and assess how missing values affect downstream analytical pipelines.
- **Version Control Mastery:** Built an solid understanding of version control concepts by setting up public repositories, staging files, tracking changes, and writing clear documentation on GitHub.
- **Exploratory Data Analysis Foundations:** Established a strong foundation in exploratory data analysis techniques, learning how to audit data quality and prepare datasets for predictive modeling.

16. Conclusion

Task 1 served as an excellent introduction to data collection and data source identification within a data analytics workflow. By exploring the historical Titanic Dataset, this project demonstrated that exploratory data analysis is much more than a preliminary step; it is a vital process for verifying data quality and ensuring the reliability of downstream machine learning models.

The programmatic exploration successfully audited the structural constraints, dimension boundaries, and feature properties of the dataset. It mapped specific column data types, quantified missing values across variables, and inspected unique category distributions. In conclusion, developing a deep, logical understanding of raw data inputs ensures that subsequent engineering, cleansing, and modeling steps are built on a solid, accurate foundation.

17. References

1. **Titanic Kaggle Dataset Reference:**
   Yasser H. (Titanic Dataset Benchmark Repository). URL: https://www.kaggle.com/datasets/yasserh/titanic-dataset
2. **Pandas Library Documentation Portal:**
   Pandas Development Team. *Pandas: Powerful Data Analysis Tools for Python*. URL: https://pandas.pydata.org/docs/
3. **Google Colab Environment Guide:**
   Google Research. *Google Colaboratory Interactive Workspace Overview*. URL: https://colab.research.google.com/
4. **GitHub Documentation Platform:**
   GitHub Inc. *GitHub Product Guides & Version Control Workflow Documentation*. URL: https://docs.github.com/